Lab 6

Task 1:

```
ubuntu@ip-172-31-64-100:~/lab6-app$ docker build -t lab6-app .
[+] Building 1.7s (10/10) FINISHED                          docker:default
 => [internal] load build definition from Dockerfile                   0.0s
 => => transferring dockerfile: 149B                                   0.0s
 => [internal] load metadata for docker.io/library/node:22             0.3s
 => [internal] load .dockerignore                                      0.0s
 => => transferring context: 2B                                        0.0s
 => [1/5] FROM docker.io/library/node:22@sha256:7e791fc54bd02fc89f     0.0s
 => => resolve docker.io/library/node:22@sha256:7e791fc54bd02fc89f     0.0s
 => [internal] load build context                                      0.1s
 => => transferring context: 42.77kB                                   0.1s
 => CACHED [2/5] WORKDIR /app                                          0.0s
 => CACHED [3/5] COPY package*.json ./                                 0.0s
 => CACHED [4/5] RUN npm install                                       0.0s
 => [5/5] COPY . .                                                     0.5s
 => exporting to image                                                 0.5s
 => => exporting layers                                                0.2s
 => => exporting manifest sha256:a71e17e5a92ce3b580d974142c0b9a763     0.0s
 => => exporting config sha256:224e0b7697652047f1fed8d0694db4eadfe     0.0s
 => => exporting attestation manifest sha256:d613ee846f19616696b41     0.0s
 => => exporting manifest list sha256:8f719779224d0113ef34cfc84a75     0.0s
 => => naming to docker.io/library/lab6-app:latest                     0.0s
 => => unpacking to docker.io/library/lab6-app:latest                  0.1s
ubuntu@ip-172-31-64-100:~/lab6-app$
```

Write-up 1: I chose **PostgreSQL.** The Docker image needs to be rebuilt after changing the application source files because the image is a snapshot of the code and environment at build time. Rebuilding ensures the updated source code and dependencies are included in the container, so the application runs with the latest changes.

Task 2:

- showing both services running:

```
ubuntu@ip-172-31-64-100:~/lab6-app$ docker compose ps
NAME              IMAGE          COMMAND                  SERVICE   CREATED          STATUS
     PORTS
lab6-app-app-1    lab6-app-app   "docker-entrypoint.s…"   app       42 seconds ago   Up 41 secon
ds   0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
lab6-app-db-1     postgres:15    "docker-entrypoint.s…"   db        15 minutes ago   Up 15 minut
es   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
ubuntu@ip-172-31-64-100:~/lab6-app$
```

- Return data from the database:

```
ubuntu@ip-172-31-64-100:~/lab6-app$ curl localhost:3000/tasks
[{"id":1,"title":"Milk"},{"id":2,"title":"Eggs"},{"id":3,"title":"Bread"}]ubuntu@ip-172-31-64
:~/lab6-app$
ubuntu@ip-172-31-64-100:~/lab6-app$
```

- showing the app started successfully:

```
ubuntu@ip-172-31-64-100:~/lab6-app$ docker logs lab6-app-app-1
--------------------------------------------------
  CISC 886 Lab 6 – App started
  Port:  3000
  Mode:  local
  Node:  v22.22.2
  Host:  d3938433c51c
--------------------------------------------------
ubuntu@ip-172-31-64-100:~/lab6-app$
```

Write-up 2:

- the output of **docker compose ps** command:

NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS

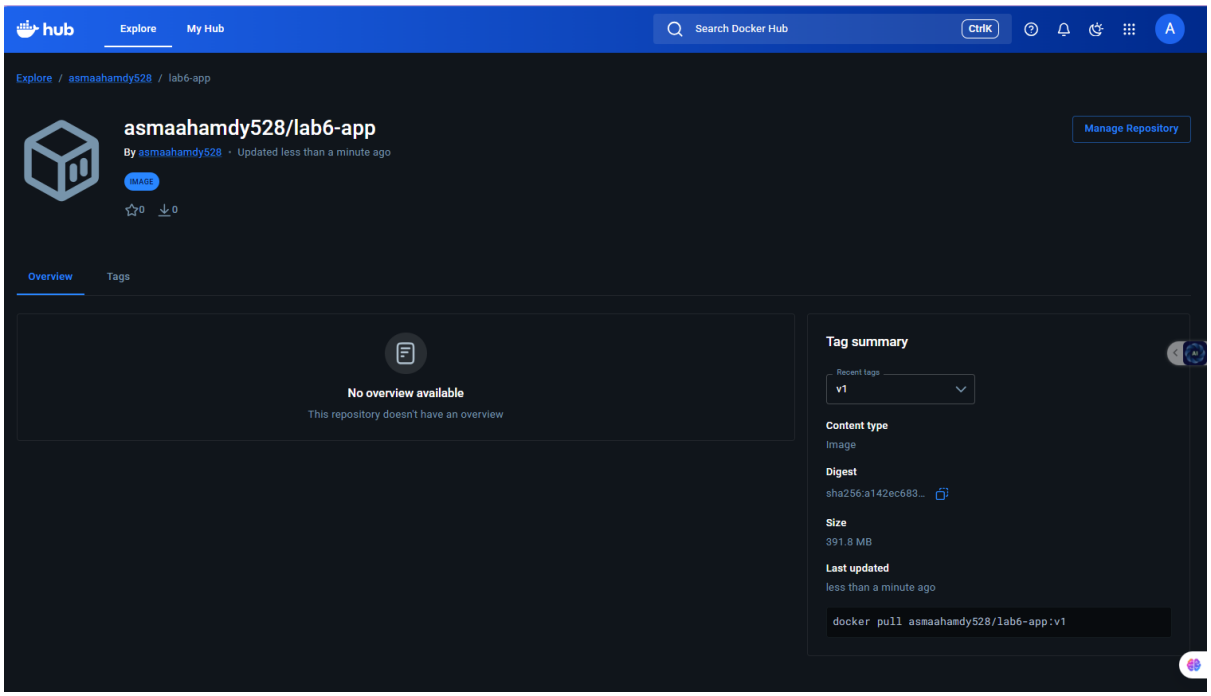
lab6-app-app-1 lab6-app-app "docker-entrypoint.s…" app 2 hours ago Up 2 hours 0.0.0.0:3000>3000/tcp, [::]:3000->3000/tcp

lab6-app-db-1 postgres:15 "docker-entrypoint.s…" db 2 hours ago Up 2 hours 0.0.0.0:5432>5432/tcp, [::]:5432->5432/tcp

- The app container can reach the database container by name because Docker Compose automatically creates a private network for the services, allowing containers to resolve each other using their service names as hostnames.

Task 3:

- Docker Hub repository page:



Write-up 3: A container registry is a centralized service for storing and distributing Docker images. It is useful in a cloud deployment workflow because it allows teams to share, version, and pull images reliably across different environments, ensuring consistent application deployment.